

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC MÁY TÍNH



MÔN HỌC

CS317.Q22 – PHÁT TRIỂN VÀ VẬN HÀNH HỆ THỐNG HỌC MÁY

BÁO CÁO ĐỒ ÁN CUỐI KỲ

**TryOps: Nền tảng MLOps cho Virtual
Try-On và LLM Serving**

Repository: github.com/trungdangtapcode/TryOps-TryON-MLOps

Giảng viên hướng dẫn: MSc. Đỗ Văn Tiến

Trợ giảng: Lê Trần Trọng Khiêm

Sinh viên:	Nguyễn Trần Nhật Trung	23521684
	Hoàng Minh Thái	23521414
	Phạm Hải Đăng	23520233

TP. Hồ Chí Minh, tháng 07 năm 2026

Tóm tắt

TryOps là một nền tảng MLOps được xây dựng quanh hai workload có yêu cầu vận hành cao: Virtual Try-On (VTON) và Large Language Model (LLM) serving. Trọng tâm của đề án không chỉ là tạo ra một kết quả inference, mà là xây dựng hệ thống xung quanh vòng đời của model: dữ liệu, artifact, endpoint cấu hình, account, quota, asynchronous job, policy gate, model registry, observability, rollback posture và các record cần thiết cho việc kiểm tra vận hành.

Hệ thống gồm bốn lớp chính. Lớp product là React/Vite console, hỗ trợ đăng nhập, workspace, VTON Studio, LLM Playground, model registry, governance, dashboard và history. Lớp edge là Rust Axum gateway, chịu trách nhiệm authentication, proxying, request limit, quota admission, guardrail dispatch, static serving và Prometheus metrics. Lớp workflow là FastAPI backend, quản lý product API, VTON job orchestration, artifact metadata, evaluation, account state và governance view. Lớp inference/control-plane gồm host-side FASHN VTON router và worker, OpenAI-compatible LLM endpoint, Go service, cùng C++ native tools cho policy, metrics, SLO check, supply-chain record, cache, energy và evaluation.

Đóng góp chính của TryOps là đặt workload AI vào một operational boundary rõ ràng. VTON cần GPU-aware job control, image artifact lineage và kiểm tra chất lượng ảnh. LLM serving cần token accounting, endpoint health, guardrail và latency/cost measurement. TryOps dùng chung một nền tảng gateway, API, storage, observability và governance cho cả hai nhóm workload.

Phiên bản hiện tại hỗ trợ developer-local stack với Postgres, Valkey, MinIO, MLflow, Keycloak, FastAPI, Rust gateway, Go guardrail, Prometheus, Grafana, Loki, Tempo, OpenTelemetry Collector và FASHN VTON router chạy phía host. Báo cáo này trình bày kiến trúc, yêu cầu, thiết kế model serving, governance, observability, port topology, benchmark VTON/LLM và các giới hạn còn lại.

Từ khóa: MLOps, Virtual Try-On, FASHN VTON, LLM Serving, vLLM, OpenAI-compatible API, Rust Gateway, FastAPI, Go, C++, Prometheus, Grafana, Loki, Tempo, MLflow, MinIO, Keycloak, governance, policy gate.

Mục lục

Tóm tắt	i
1 Giới thiệu	1
1.1 Động lực	1
1.2 Mục tiêu đề án	1
1.3 Hướng phát triển	2
1.4 Đặc điểm kỹ thuật	3
1.5 Technology Stack	3
2 Cơ sở kỹ thuật và yêu cầu MLOps	4
2.1 MLOps trong hệ thống AI	4
2.2 Virtual Try-On	4
2.3 LLM Serving	5
3 Phân tích yêu cầu	6
3.1 Actor	6
3.2 Yêu cầu chức năng	6
3.2.1 Module TryOps Console và Workspace	6
3.2.2 Module VTON Workflow	7
3.2.3 Module LLM Serving	7
3.2.4 Module Governance và Registry	8
3.2.5 Module Observability và Operations	8
3.3 Yêu cầu phi chức năng	9
3.4 Luồng xử lý chính	9
3.4.1 Luồng Virtual Try-On	10
3.4.2 Luồng LLM Request	10
3.4.3 Luồng Model Promotion	10

3.5	Ranh giới Product và Diagnostics	10
4	Kiến trúc hệ thống	12
4.1	Tổng quan kiến trúc	12
4.2	Frontend Layer	13
4.3	Gateway Layer	14
4.4	API và Workflow Layer	15
4.5	Data và Artifact Layer	15
4.6	Observability Layer	16
5	Thiết kế Model Serving	18
5.1	VTON Serving	18
5.2	VTON Job Lifecycle	19
5.3	Multi-GPU Worker Configuration	19
5.4	LLM Serving	20
6	Governance, Security và Responsible AI	21
6.1	Authentication và Workspace Model	21
6.2	Quota và Admission Control	21
6.3	Guardrail	22
6.4	Promotion Gate	22
6.5	Supply Chain và Privacy	23
7	Observability và Vận hành	24
7.1	Dashboard	24
7.2	Logs	24
7.3	Metrics	24
7.4	Developer-Local Ports	25
7.5	Deployment Preconditions	28
8	Thực nghiệm và đánh giá	29
8.1	VTON Benchmark Results	29
8.2	LLM Optimization Results	30
8.3	Native Boundary Evaluation	30
8.4	Balancer Testing	31

9	Giới hạn và hướng phát triển	32
9.1	Giới hạn hiện tại	32
9.2	Hướng phát triển	33
10	Kết luận	34
	Tài liệu tham khảo	34

Danh sách hình

1.1	Technology stack của TryOps theo từng lớp sản phẩm.	3
4.1	Product boundary architecture của TryOps.	12
4.2	Layered architecture của TryOps.	12
4.3	Giao diện TryOps Studio	13
4.4	Workspace và quota view	14
4.5	VTON job đang chạy trong UI	14
4.6	MinIO artifact store	16
4.7	MLflow benchmark tracking	16
4.8	Grafana observability dashboard	17
4.9	Log drilldown dashboard để debug job và request	17
5.1	FASHN VTON v1.5 model flow	18
6.1	Keycloak session control và identity surface	21
7.1	Developer-local service topology và quan hệ giữa host port, Compose network và host inference runtime.	26
8.1	Diễn giải benchmark VTON và so sánh average latency.	30
9.1	VTON optimization limitation	33

1. Giới thiệu

1.1 Động lực

Nhiều ứng dụng học máy có thể tạo ra một ảnh kết quả hoặc một đoạn văn trả lời có vẻ thuyết phục. Tuy nhiên, một hệ thống ML được vận hành nghiêm túc phải trả lời được các câu hỏi ngoài phạm vi notebook:

- Dữ liệu, model weights, code version, configuration và phần cứng nào đã tạo ra output?
- Candidate model có bị chặn trước khi tới người dùng nếu không đạt policy hay không?
- Khi endpoint đã cấu hình không sẵn sàng, hệ thống có trả operational error rõ ràng không?
- Mỗi workspace có được kiểm soát bằng quota, rate limit và concurrency limit không?
- Operator có debug được lỗi bằng metrics, logs, traces, job ID và request ID không?
- Release có thể rollback và giải thích lại cho reviewer không?

TryOps được thiết kế để trả lời các câu hỏi trên. Hai workload được chọn có tính chất khác nhau: VTON là workload thị giác, nặng GPU, nhạy về quyền riêng tư ảnh người dùng và khó đánh giá vì lỗi thường mang tính cảm nhận; LLM serving nhạy với latency, throughput, token cost, guardrail và prompt injection. Nếu một nền tảng có thể vận hành cả hai workload này trong cùng một governance spine, nền tảng đó thể hiện rõ tư duy MLOps hơn một ứng dụng đơn lẻ.

1.2 Mục tiêu đề án

Mục tiêu của đề án là xây dựng một nền tảng MLOps có thể vận hành workload AI với traceability, observability, quota và governance.

Các mục tiêu cụ thể:

1. Xây dựng TryOps Console cho end user, operator và reviewer.
2. Dùng Rust gateway làm edge boundary.
3. Dùng FastAPI làm product workflow backend và model adapter layer.

4. Chạy VTON qua FASHN VTON serving path.
5. Hỗ trợ LLM inference qua OpenAI-compatible endpoint, bao gồm self-hosted vLLM hoặc managed API.
6. Tích hợp Keycloak cho OIDC login và workspace identity.
7. Tích hợp Postgres, Valkey, MinIO và MLflow cho product data, counter, object storage và experiment tracking.
8. Tích hợp Prometheus, Grafana, Loki, Tempo và OpenTelemetry cho observability.
9. Cài đặt governance thông qua policy gate, supply-chain record, model card, data card và promotion record.
10. Tách bạch utility cho testing/evaluation khỏi product inference path.

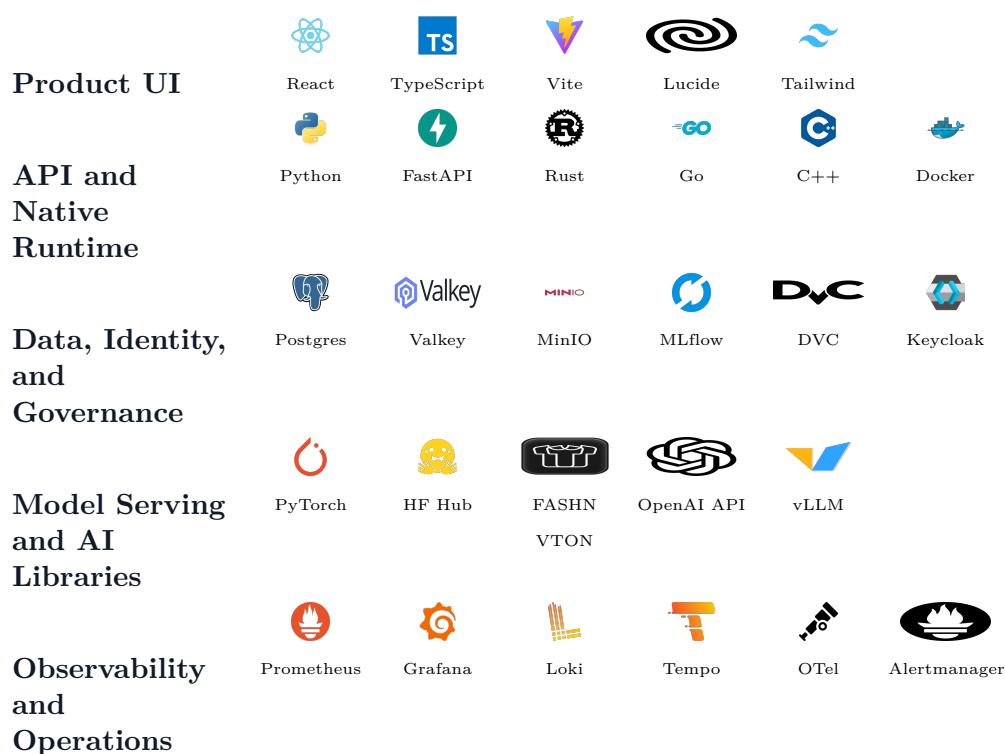
1.3 Hướng phát triển

Nhóm	Hiện tại	Tương lai
Product	TryOps Console, LLM Playground, VTON Studio, Account, Registry, Governance, Dashboard, History	Bổ sung accessibility, export workflow và nhiều operator action hơn
VTON	FASHN VTON router, GPU worker, image upload, async job, artifact output, metrics	Human preference study và model zoo rộng hơn
LLM	OpenAI-compatible endpoint, OpenAI/vLLM configuration, guardrail, quota	Live vLLM benchmark phụ thuộc workspace và GPU khả dụng
MLOps	MLflow, MinIO, promotion record, model/data card, policy gate	KServe/Kubeflow deployment profile
Observability	Prometheus, Grafana, Loki, Tempo, OTel bridge, structured logs	GPU/process exporter đầy đủ cho nhiều biến thể shared-datacenter
Security	Keycloak, gateway auth, quota, guardrail, supply-chain record	External Secrets/Vault cho production cluster
Networking	Developer-local port map và shared-datacenter plan	Ingress/hardening profile cuối cùng cho multi-tenant data center

1.4 Đặc điểm kỹ thuật

- Một local deployment path có thể triển khai cho full product stack trên GCP hoặc AWS.
- Native-first boundary: Rust ở edge, Go cho service/controller/load tool, C++ cho native tools, Python cho ML workflow.
- VTON serving abstraction: API gọi một endpoint ổn định, router ẩn worker count, GPU mapping, socket và per-worker metrics.
- LLM adapter dùng OpenAI-compatible API và endpoint-error handling rõ ràng.
- Observability stack gồm Grafana dashboard, Loki logs, Tempo traces, Prometheus metrics và structured job/request logs.
- Governance pipeline gồm policy gate, supply-chain record, dependency lock, model scan, provenance và promotion lifecycle record.

1.5 Technology Stack



Hình 1.1: Technology stack của TryOps theo từng lớp sản phẩm.

2. Cơ sở kỹ thuật và yêu cầu MLOps

2.1 MLOps trong hệ thống AI

MLOps là tập hợp quy trình và công cụ để đưa model từ giai đoạn thử nghiệm sang vận hành có kiểm soát. Trong đề án này, các yêu cầu MLOps quan trọng gồm:

- Versioning cho data, code, configuration và model artifact.
- Experiment tracking và artifact storage.
- Evaluation trước khi promotion.
- Model registry với lifecycle state: candidate, challenger, champion, rejected, archived.
- CI/CD và policy gate.
- Observability cho metrics, logs, traces, drift, quality và cost.
- Security, quota, guardrail và rollback.

TryOps hiện thực các yêu cầu trên bằng MLflow, MinIO, Postgres, Valkey, generated reports, runtime artifacts, native policy/evaluation tools và Grafana/Loki/Tempo dashboards.

2.2 Virtual Try-On

Virtual Try-On nhận ảnh người và ảnh trang phục, sau đó sinh ảnh người mặc trang phục đó. Đây là workload khó vận hành vì:

- Output là ảnh, lỗi có thể là méo hình, identity drift, mất texture, sai pose, hỏng chi tiết cơ thể hoặc artifact tại vùng biên trang phục.
- GPU memory và latency có thể lớn.
- Ảnh người dùng có tính riêng tư cao.
- Hệ thống phải lưu input, output, report metadata, metrics và lineage.
- Hệ thống phải biết output được tạo từ product path hay từ evaluation utility.

TryOps sử dụng FASHN VTON v1.5 cho VTON serving. Theo tài liệu nghiên cứu của FASHN, VTON v1.5 là maskless pixel-space virtual try-on model, hướng tới việc giữ chi tiết trang phục mà không cần segmentation mask thủ công [10].

2.3 LLM Serving

LLM serving cần theo dõi latency, throughput, token count, token cost, quota, cache effect, guardrail outcome và endpoint failure. TryOps chuẩn hóa LLM serving thông qua OpenAI-compatible API. Endpoint có thể là self-hosted vLLM hoặc managed provider, nhưng product contract vẫn giống nhau: request đi qua gateway, backend ghi lại metadata cần thiết, provider secret được đưa vào runtime configuration, và lỗi endpoint được đưa vào observability stack.

3. Phân tích yêu cầu

3.1 Actor

Actor	Nhu cầu
End user	Login, upload ảnh, chạy try-on, xem trạng thái job, mở kết quả
ML engineer	Chạy benchmark, so sánh candidate, xem evaluation report
Platform engineer	Vận hành stack, xem logs/metrics/traces, cấu hình end-point và port
Risk reviewer	Kiểm tra model card, data card, policy gate, supply-chain record và lineage
Admin/operator	Theo dõi quota, account, registry state, incident và rollback record
Evaluator/instructor	Kiểm tra behavior qua code, dashboard, artifact, report và kết quả benchmark

3.2 Yêu cầu chức năng

Các yêu cầu chức năng được chia theo module để làm rõ trách nhiệm của từng phần trong hệ thống. Cách đánh mã FR-* giúp liên kết yêu cầu với thiết kế, kiểm thử, dashboard và phần đánh giá ở các chương sau.

3.2.1 Module TryOps Console và Workspace

Bảng 3.2: Yêu cầu chức năng – Module TryOps Console và Workspace.

ID	Yêu cầu	Mô tả
FR-UI-01	Đăng nhập và session	User đăng nhập qua Keycloak/OIDC; frontend nhận trạng thái session và gửi request qua Rust gateway.
FR-UI-02	Workspace context	UI hiển thị workspace hiện tại, account name, plan/quota và trạng thái sử dụng tài nguyên.

ID	Yêu cầu	Mô tả
FR-UI-03	Điều hướng sản phẩm	User chuyển được giữa Studio, Account, Registry, Governance, Dashboard và History.
FR-UI-04	Job visibility	UI hiển thị active, queued, completed và failed job; user có thể xem lại output và trạng thái job sau khi thao tác.
FR-UI-05	Feedback và history	User có thể xem request history, job history và gửi feedback cho kết quả model.

3.2.2 Module VTON Workflow

Bảng 3.3: Yêu cầu chức năng – Module VTON Workflow.

ID	Yêu cầu	Mô tả
FR-VTON-01	Upload ảnh đầu vào	User upload person image và garment image; API kiểm tra định dạng, lưu artifact và trả meta-data cho UI.
FR-VTON-02	Cấu hình try-on	User chọn garment category, source type và quality mode trước khi submit job.
FR-VTON-03	Submit async job	Backend tạo VTON job bất đồng bộ, kiểm tra quota/concurrency và lưu job snapshot.
FR-VTON-04	Theo dõi job status	UI poll trạng thái accepted , queued , running , completed hoặc failed .
FR-VTON-05	Gọi model serving	VTON adapter gửi request tới FASHN router endpoint đã cấu hình; router dispatch tới GPU worker.
FR-VTON-06	Lưu output artifact	Khi job hoàn tất, hệ thống lưu output image, report metadata, latency và lineage.
FR-VTON-07	Hiển thị kết quả	UI hiển thị generated image, job duration, trạng thái và link artifact cho user.

3.2.3 Module LLM Serving

Bảng 3.4: Yêu cầu chức năng – Module LLM Serving.

ID	Yêu cầu	Mô tả
FR-LLM-01	Gửi prompt	User gửi prompt từ LLM Playground; request đi qua gateway, guardrail và FastAPI backend.
FR-LLM-02	Gọi OpenAI-compatible endpoint	Backend gọi configured OpenAI-compatible endpoint, bao gồm managed provider hoặc self-hosted vLLM.

ID	Yêu cầu	Mô tả
FR-LLM-03	Ghi metadata	Hệ thống ghi model name, provider, token count, latency, tokens/s và cost estimate khi có cấu hình.
FR-LLM-04	Guardrail outcome	Guardrail có thể allow, redact, review hoặc block request/response theo policy.
FR-LLM-05	Lưu request history	Prompt metadata, response metadata và trạng thái được lưu để user/operator xem lại.

3.2.4 Module Governance và Registry

Bảng 3.5: Yêu cầu chức năng – Module Governance và Registry.

ID	Yêu cầu	Mô tả
FR-GOV-01	Model registry view	Operator xem model lifecycle state như candidate, challenger, champion, rejected và archived.
FR-GOV-02	Model card và data card	Reviewer xem metadata về model source, dataset, owner, intended use và risk note.
FR-GOV-03	Promotion gate	Hệ thống lưu policy verdict, evaluation report, SBOM, scan result, provenance và approval record.
FR-GOV-04	Rollback record	Operator xem candidate/champion history để phục vụ rollback và audit.
FR-GOV-05	Supply-chain record	Hệ thống lưu dependency lock, model-source pin và artifact provenance cho release.

3.2.5 Module Observability và Operations

Bảng 3.6: Yêu cầu chức năng – Module Observability và Operations.

ID	Yêu cầu	Mô tả
FR-OPS-01	Service dashboard	Grafana hiển thị request rate, error rate, latency, memory, queue depth và gateway errors.
FR-OPS-02	Log drilldown	Operator tìm log theo job ID, request ID, account ID, worker ID và GPU ID.
FR-OPS-03	Trace và metrics	OpenTelemetry, Tempo và Prometheus ghi trace/metric cho API, gateway, router và worker.
FR-OPS-04	Alerting	Prometheus/Alertmanager hỗ trợ rule cho service health, error rate và latency.

ID	Yêu cầu	Mô tả
FR-OPS-05	Startup và port map	Hệ thống có developer-local port table, health check và startup output để operator biết entry point.

3.3 Yêu cầu phi chức năng

Bảng 3.7: Yêu cầu phi chức năng của TryOps.

ID	Yêu cầu	Tiêu chí
NFR-01	Security	User request đi qua Keycloak/OIDC và Rust gateway; development shortcut không được dùng cho deployment nghiêm túc.
NFR-02	Privacy	Logs không lưu raw prompt, raw uploaded image hoặc provider secret; log dùng job ID, request ID và artifact ID.
NFR-03	Observability	Mỗi workflow quan trọng có metrics, logs và traces; Grafana hỗ trợ drilldown cho job lỗi.
NFR-04	Endpoint readiness	VTON/LLM adapter phải báo lỗi rõ khi configured endpoint không sẵn sàng để operator xử lý hạ tầng.
NFR-05	Scalability	FASHN router hỗ trợ nhiều GPU worker, per-worker metrics và routing theo in-flight state.
NFR-06	Performance	Benchmark ghi average latency, P95 latency, throughput, GPU utilization và VRAM để so sánh candidate.
NFR-07	Reproducibility	Model/data/config/code/artifact có record; evaluation và promotion report có thể truy vết lại.
NFR-08	Operability	Stack có health check, startup output, port map, dashboard và log drilldown để debug trong vận hành.
NFR-09	Shared-host networking	Public surface nên thu hẹp về gateway, Keycloak và admin-restricted Grafana; internal services nằm trong private network.
NFR-10	Governance	Promotion cần model card, data card, policy verdict, scan result, provenance và owner approval.

3.4 Luồng xử lý chính

3.4.1 Luồng Virtual Try-On

1. User đăng nhập và mở TryOps Studio trong workspace.
2. User upload person image và garment image.
3. UI gửi metadata cấu hình gồm garment category, source type và quality mode.
4. Backend kiểm tra authentication, quota và workspace concurrency.
5. Backend tạo VTON job bất đồng bộ và lưu job snapshot.
6. Runner gọi FASHN router endpoint để thực hiện model serving.
7. Router dispatch request tới GPU worker phù hợp.
8. Worker sinh output image; backend lưu output artifact, report metadata và lineage.
9. UI poll job status và hiển thị kết quả khi job completed.

3.4.2 Luồng LLM Request

1. User nhập prompt từ LLM Playground.
2. Gateway kiểm tra identity, rate limit và quota admission.
3. Guardrail đánh giá request theo policy.
4. Backend gọi OpenAI-compatible endpoint đã cấu hình.
5. Hệ thống ghi latency, token count, model metadata và guardrail outcome.
6. UI hiển thị response hoặc operational error tương ứng.

3.4.3 Luồng Model Promotion

1. ML engineer tạo candidate từ benchmark/evaluation run.
2. Hệ thống lưu artifact, model card, data card và evaluation report.
3. Policy gate kiểm tra quality, security, provenance và approval.
4. Candidate đạt yêu cầu được chuyển sang challenger hoặc champion.
5. Candidate không đạt yêu cầu được ghi rejection record để reviewer kiểm tra.

3.5 Ranh giới Product và Diagnostics

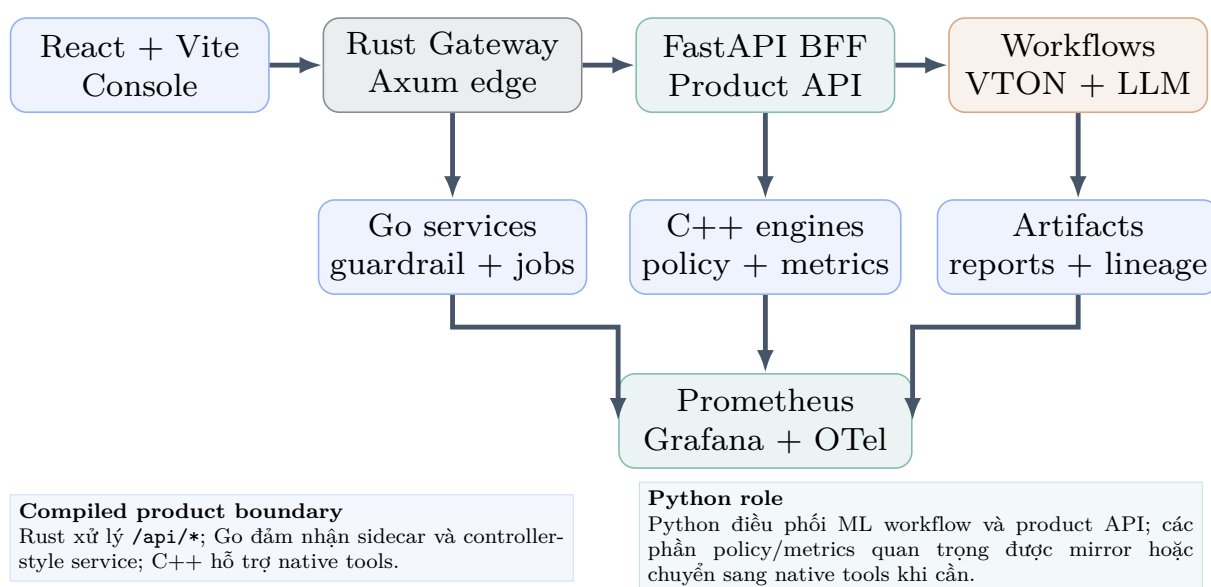
Component	Product path	Diagnostics/evaluation utility
-----------	--------------	--------------------------------

VTON	FASHN router qua configured VTON serving endpoint	Overlay/image utility chỉ dùng khi bật rõ cho test
LLM	OpenAI-compatible model endpoint	Harness cho adapter check
Semantic cache	Feature có cấu hình và có quan sát	Tắt trong cấu hình ưu tiên an toàn
API keys	OIDC/JWT và provider secret	Static development key tắt trong cấu hình mặc định
Worker ports	Ẩn sau router	Direct worker port chỉ dùng để debug

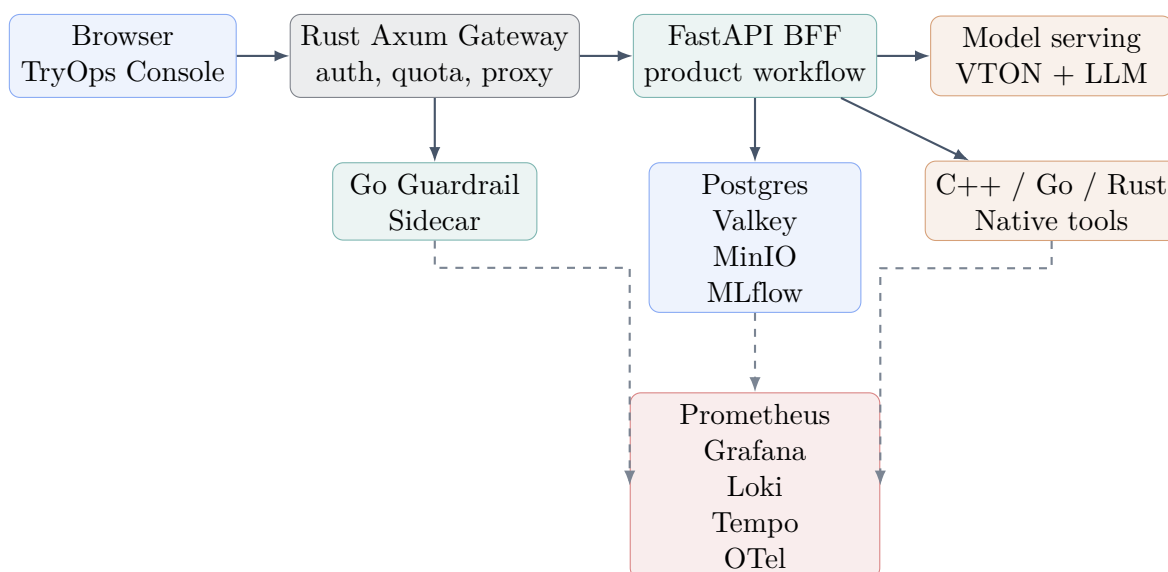
Ranh giới này giúp báo cáo đúng năng lực của hệ thống: phần product path phục vụ user, còn diagnostics/evaluation utility phục vụ kiểm thử, đo đạc và phân tích.

4. Kiến trúc hệ thống

4.1 Tổng quan kiến trúc



Hình 4.1: Product boundary architecture của TryOps.

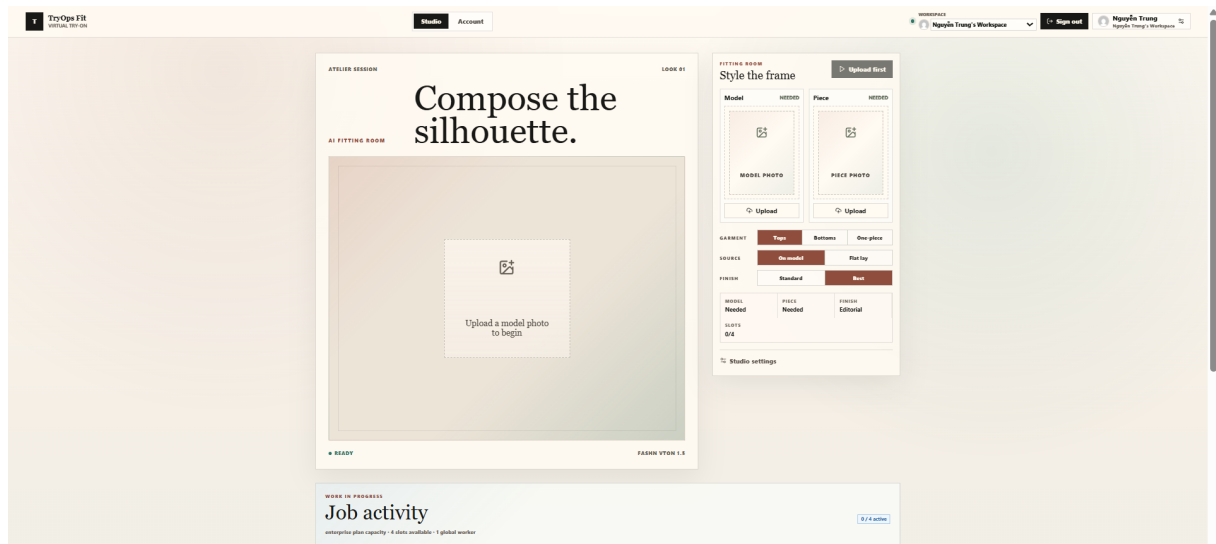


Hình 4.2: Layered architecture của TryOps.

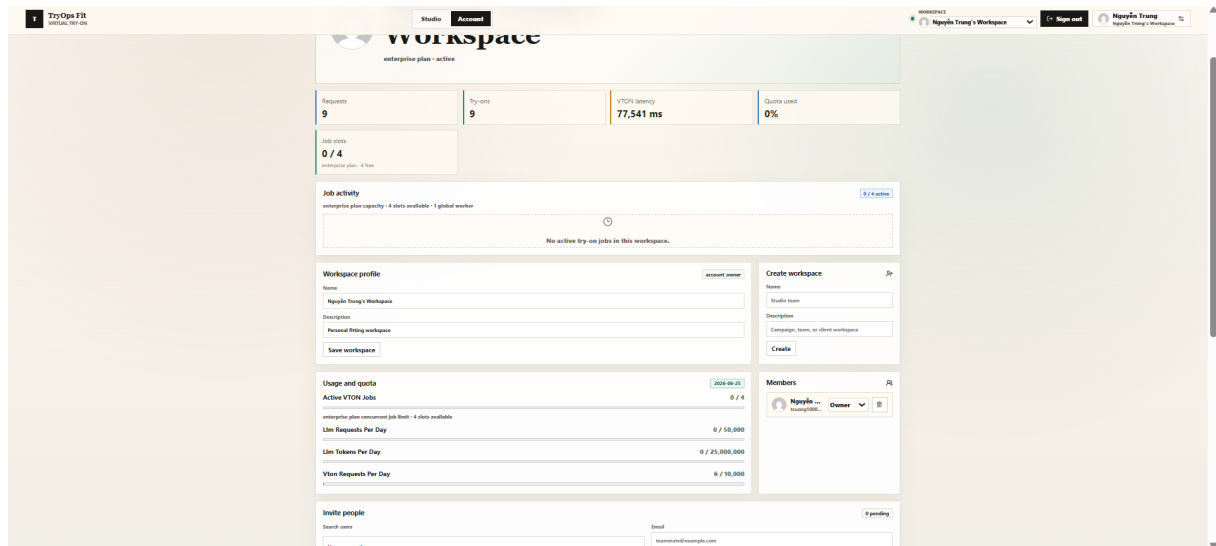
4.2 Frontend Layer

Frontend được tổ chức theo workflow sản phẩm, không chỉ là trang thử nghiệm đơn lẻ. Các chức năng chính:

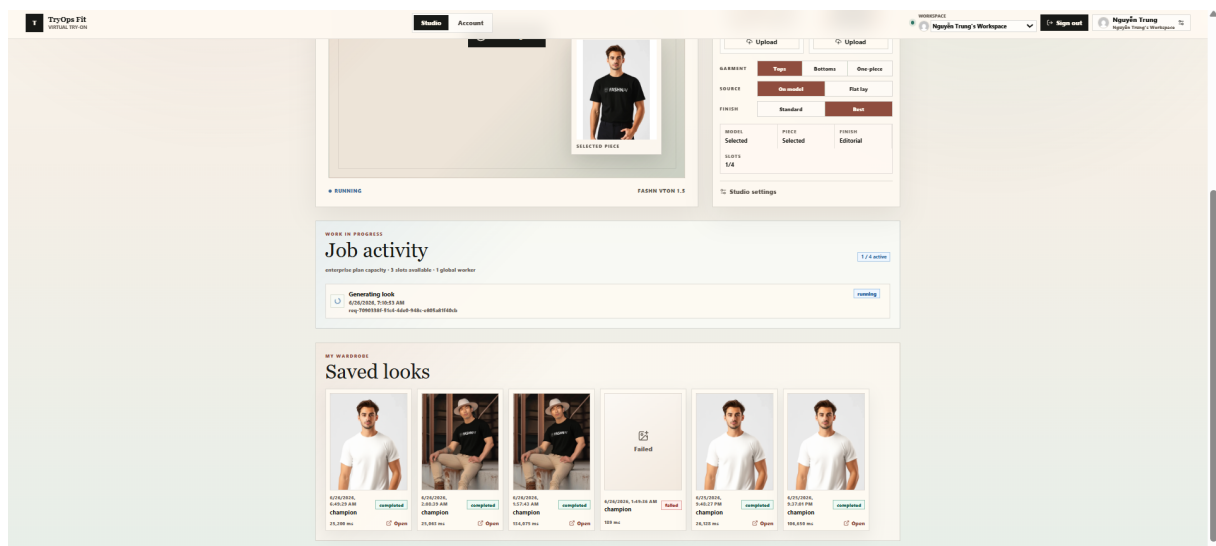
- VTON image upload, garment/source/quality selection và async job submission.
- Trình bày active, queued, completed và failed job.
- Gallery cho VTON output gần đây.
- LLM prompt workflow.
- Workspace quota và account state.
- Operator views cho dashboard, governance, registry, incident và rollback record.
- OIDC authorization-code flow với PKCE.



Hình 4.3: Giao diện TryOps Studio.



Hình 4.4: Workspace và quota view.



Hình 4.5: VTON job đang chạy trong UI.

4.3 Gateway Layer

Rust gateway là edge boundary của hệ thống:

- Serve static UI assets.
- Proxy product API request tới workflow backend.
- Validate API key hoặc JWT/OIDC identity.
- Enforce rate limit và payload limit.
- Thực hiện quota admission trước khi chuyển tiếp workload tốn chi phí.
- Gọi Go guardrail sidecar trước LLM request.

- Forward request ID và trace context.
- Expose Prometheus metrics.
- Hỗ trợ TLS-oriented deployment profile.

Thiết kế này giúp Python không phải là security và quota boundary duy nhất.

4.4 API và Workflow Layer

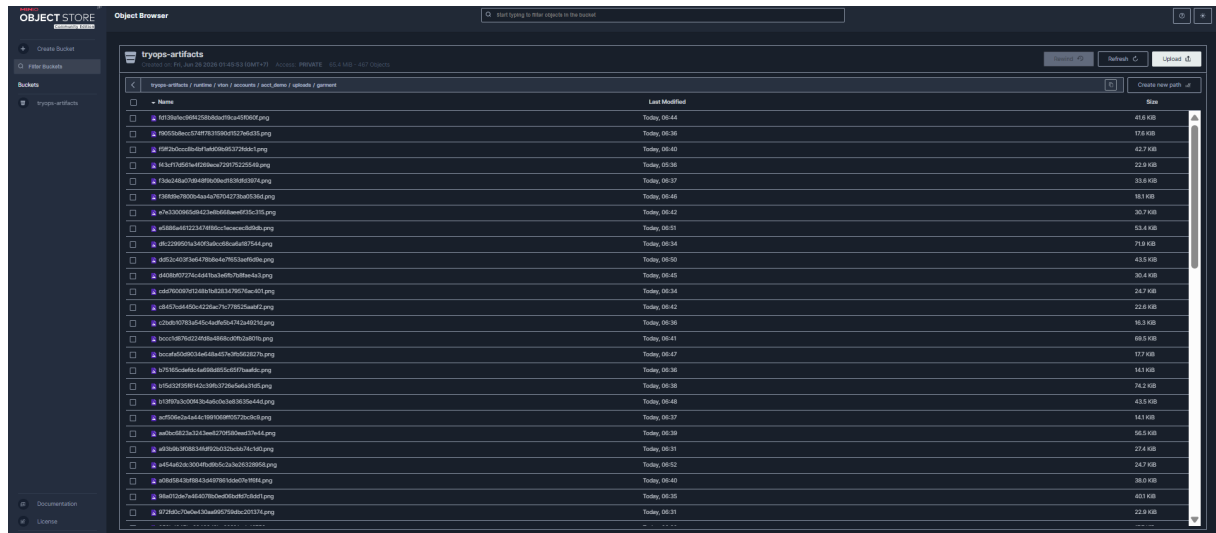
FastAPI backend quản lý product workflow:

- VTON image upload và async job management.
- LLM generation.
- Account/session API.
- Artifact read/write.
- Dashboard rollup.
- Request history và feedback.
- Promotion, evaluation, governance và lineage endpoints.
- Observability recording.

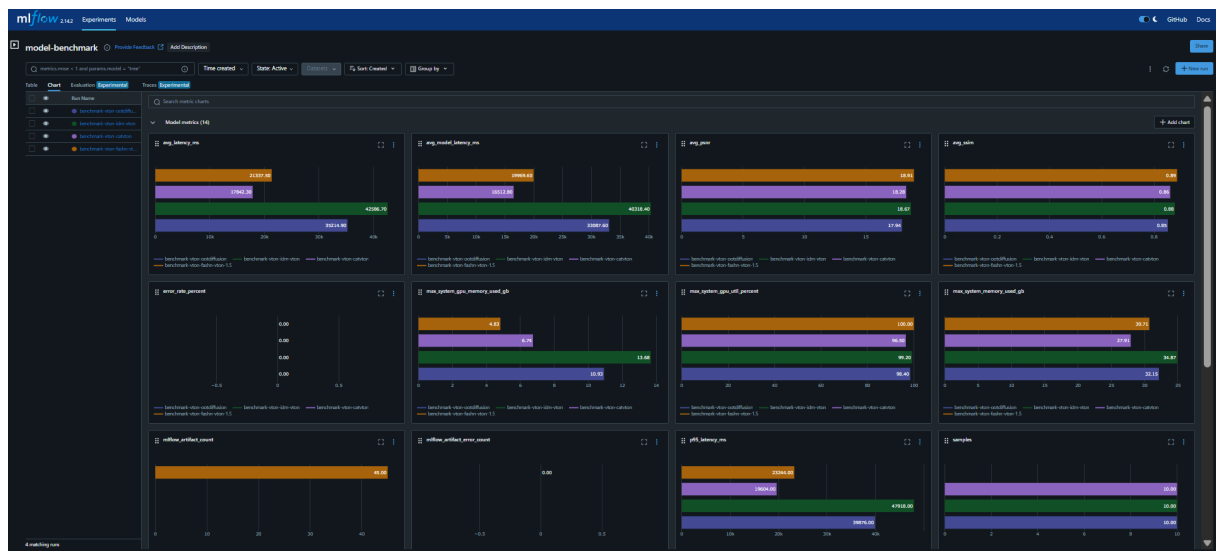
FastAPI không trực tiếp quản lý GPU scheduling. Nó gọi model serving thông qua URL ổn định.

4.5 Data và Artifact Layer

Component	Vai trò
Postgres	Account, account member, request, job, quota ledger, artifact metadata
Valkey	Hot counter cho quota và rate limiting
MinIO	Object storage cho upload, output và MLflow artifact
MLflow	Experiment tracking và model registry record
DVC	Dataset và sample data versioning
Runtime artifact repository	Logs, reports, model weights, generated outputs
Generated reports repository	Promotion và evaluation reports



Hình 4.6: MinIO artifact store.



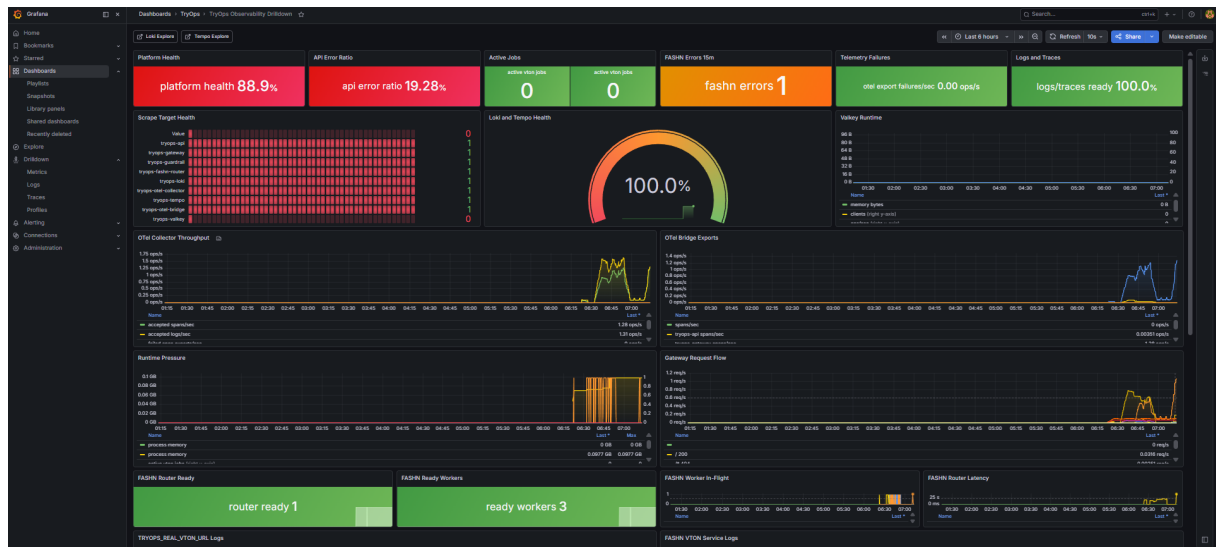
Hình 4.7: MLflow benchmark tracking.

4.6 Observability Layer

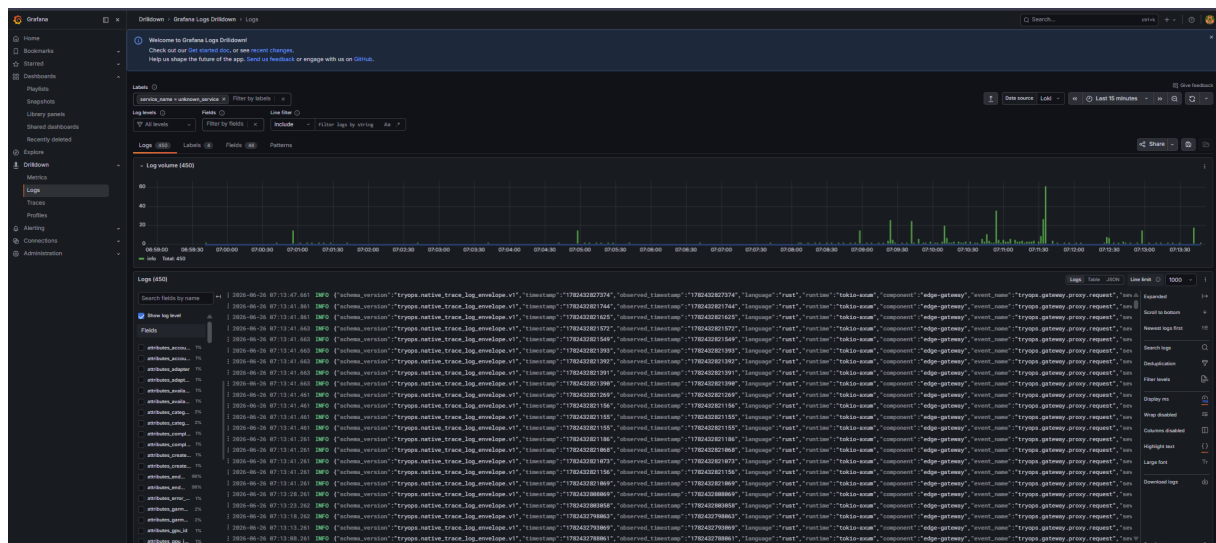
Telemetry source gồm:

- FastAPI service metrics.
- Gateway service metrics.
- FASHN router và worker metrics.
- Go guardrail metrics.
- Structured API, router và worker logs.
- Trace span cho API và background job execution.

- OTel Collector và bridge components để export logs/traces tới Loki và Tempo.



Hình 4.8: Grafana observability dashboard.



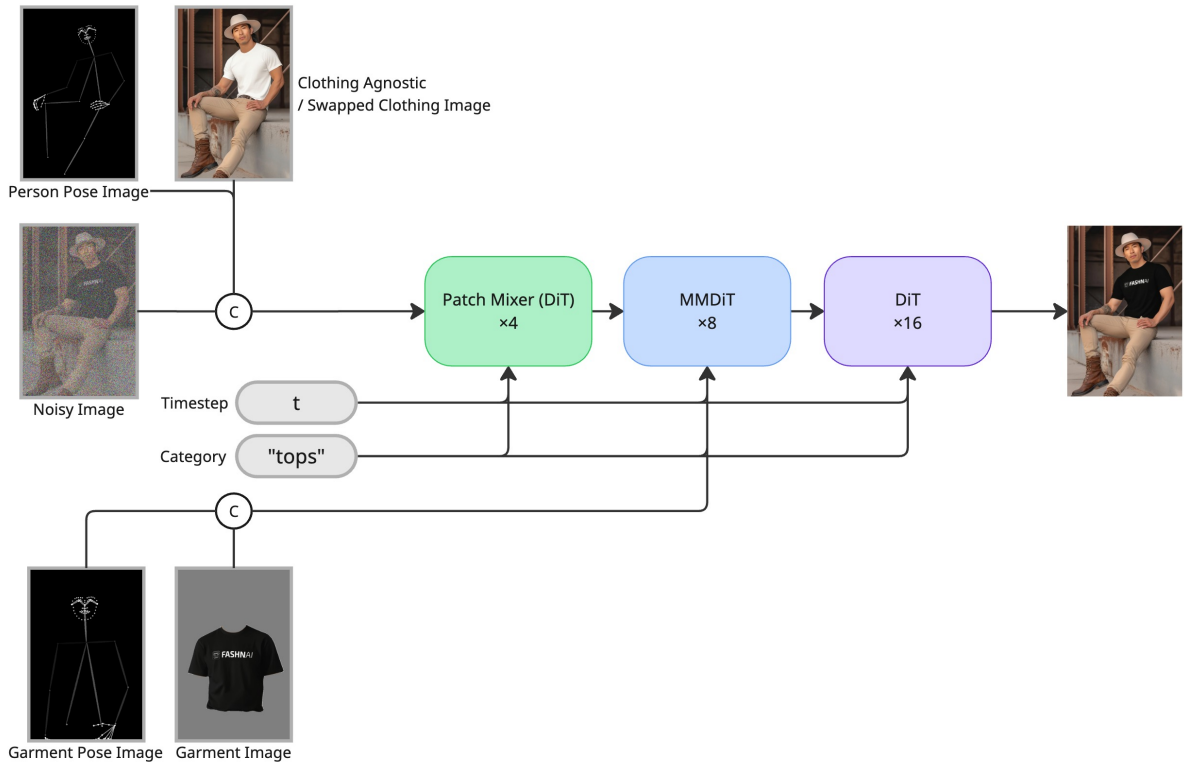
Hình 4.9: Log drilldown dashboard để debug job và request.

5. Thiết kế Model Serving

5.1 VTON Serving

VTON serving được tổ chức theo một serving path nhiều lớp. Product API gửi inference request tới một router endpoint ổn định. Router phân phối việc cho các FASHN worker process chạy phía host và được pin vào CUDA-capable GPU. Cách này che GPU topology khỏi product API, nhưng vẫn giữ per-worker metrics để operator quan sát.

Ở cấp model, FASHN VTON kết hợp person pose, clothing-agnostic person representation, garment pose, garment image, noisy image state, timestep và garment category. Các tín hiệu này đi qua Patch Mixer, MMDiT và DiT blocks để tạo ảnh try-on cuối cùng.



Hình 5.1: FASHN VTON v1.5 model flow. Pose, garment conditioning, timestep và category information đi qua Patch Mixer, MMDiT và DiT blocks trước khi tạo ảnh try-on. Source: FASHN VTON v1.5 research page.

FASHN router là abstraction boundary:

- Đọc GPU assignment từ configuration.
- Tạo worker definition theo từng GPU được cấu hình.
- Pin mỗi worker vào một CUDA device.
- Dùng Unix socket để tránh mở public worker port.
- Thực hiện readiness check.
- Route request bằng least-inflight kết hợp round-robin.
- Expose Prometheus metrics với label `worker_id`, `gpu_id`, `gpu_uuid` và `status`.
- Ghi structured router và worker logs.

5.2 VTON Job Lifecycle

1. User upload person image và garment image.
2. API normalize và lưu image artifacts.
3. User submit VTON job.
4. API kiểm tra payload, auth, quota và workspace concurrency.
5. VTON job queue lưu job snapshot.
6. Runner gọi VTON inference implementation.
7. Adapter gọi FASHN router endpoint.
8. FASHN worker tạo output PNG và report metadata.
9. API lưu request record và job result.
10. UI poll job status và mở output artifact.

Status	Ý nghĩa
accepted	API nhận job
queued	Job đang chờ trong queue
running	Job đang thực thi
completed	Job có output artifact
failed	Job lỗi và có error message

5.3 Multi-GPU Worker Configuration

Trong multi-GPU deployment, router được cấu hình bằng danh sách GPU identifier có thể sử dụng. Mỗi worker gắn với một GPU và giao tiếp qua private endpoint. Product API chỉ gọi router, còn Prometheus/Grafana quan sát worker behavior qua label về worker identity, GPU identity, request status, latency và in-flight count.

Worker	CUDA pin	Private endpoint
fashn-gpu0	CUDA_VISIBLE_DEVICES=0	Unix socket
fashn-gpu1	CUDA_VISIBLE_DEVICES=1	Unix socket
fashn-gpu2	CUDA_VISIBLE_DEVICES=2	Unix socket

API và Prometheus không cần biết private worker endpoint. Chúng giao tiếp qua router. Grafana vẫn hiển thị được per-worker activity vì router phát metrics có worker và GPU labels.

5.4 LLM Serving

LLM serving dùng OpenAI-compatible adapter. Adapter gọi chat-completions interface và ghi lại:

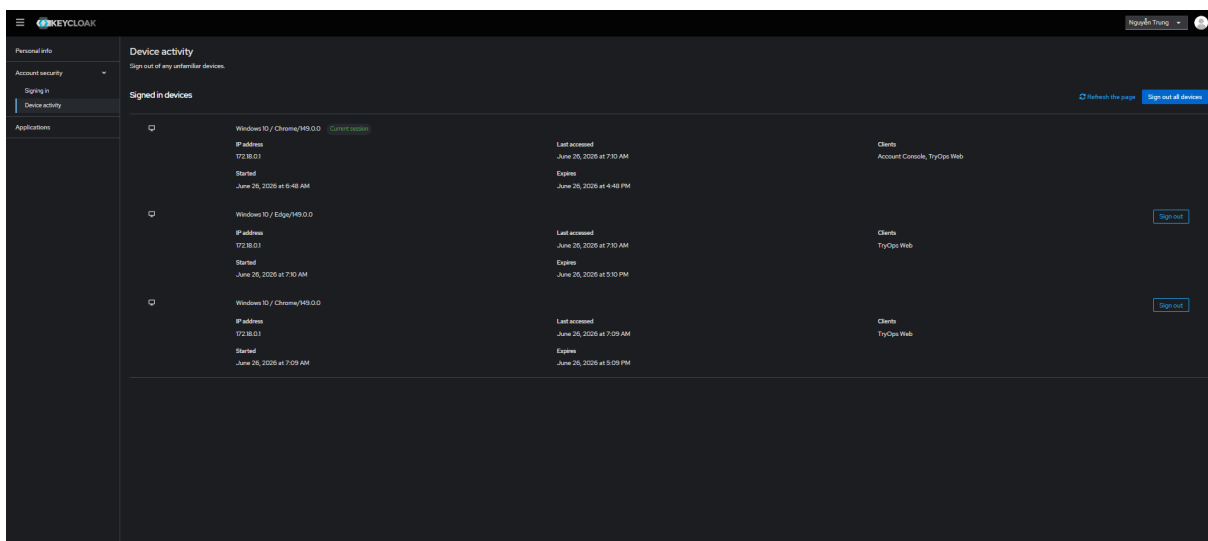
- model name,
- provider/base URL sau khi sanitize,
- input và output token count,
- latency,
- tokens per second,
- estimated cost khi có cấu hình,
- guardrail flags,
- structured output validation khi request yêu cầu.

Thiết kế này cho phép thay đổi provider hoặc self-hosted endpoint mà không làm thay đổi product workflow.

6. Governance, Security và Responsible AI

6.1 Authentication và Workspace Model

TryOps dùng Keycloak làm OIDC identity provider. Browser đăng nhập bằng authorization code flow với PKCE. Gateway validate token và forward principal metadata tới backend. Khi user đăng nhập lần đầu, backend bootstrap workspace account trong Postgres.



Hình 6.1: Keycloak session control và identity surface.

6.2 Quota và Admission Control

Quota được kiểm soát ở hai lớp:

- Rust gateway thực hiện edge quota/rate admission.
- FastAPI backend thực hiện workload-level VTON workspace concurrency.

Plan	Active VTON jobs
free	1
team	2
enterprise	4

Plan limit khác với GPU worker count. Một workspace có thể bị giới hạn một active job dù router có nhiều GPU worker.

6.3 Guardrail

Guardrail chạy tại gateway và API boundary:

- Prompt injection detection.
- System prompt leakage prevention.
- PII và secret-like string masking.
- Output size và structured output validation.
- Metrics theo risk category.

Go guardrail service được gọi qua internal service URL đã cấu hình.

6.4 Promotion Gate

Model promotion yêu cầu:

- model card,
- data card,
- evaluation report,
- SBOM,
- model artifact scan,
- provenance attestation,
- risk controls,
- owner approvals,
- policy verdict.

State	Ý nghĩa
candidate	Output từ pipeline, chưa được tin cậy để promote
challenger	Đã qua staging gate, có thể shadow/canary
champion	Model đang được promote
rejected	Bị policy chặn
archived	Giữ lại cho audit và rollback history

6.5 Supply Chain và Privacy

Repository chứa tài liệu về model source, dependency inventory, dataset licensing và promotion decision. Production logs không nên lưu raw prompt, raw uploaded image hoặc provider secret. Logs nên tham chiếu artifact ID, request ID, trace ID và job ID để hỗ trợ debugging mà vẫn giữ privacy.

7. Observability và Vận hành

7.1 Dashboard

Dashboard	Mục đích
TryOps Service Overview	Request rate, error ratio, latency, memory, queue depth, gateway errors
TryOps Model Quality	Quality score, VTON latency, LLM throughput, completed inference
TryOps Cost and Capacity	Cost, quota utilization, cache hit rate, energy/carbon
TryOps Guardrails	Blocked/redacted/reviewed findings theo risk category
TryOps Observability Drill-down	Logs, job lifecycle, FASHN service logs, OTel/Tempo/Loki health

7.2 Logs

Các nhóm log quan trọng:

- API request và job lifecycle events.
- FASHN router events.
- Per-worker VTON inference events.
- Model-serving process logs.
- Trace spans cho API và background jobs.

Grafana nên cho phép tìm log theo job ID, request ID, account ID, worker ID và GPU ID. Điều này cần thiết cho các tình huống UI đang chờ nhưng backend đã hoàn thành hoặc đã ghi lỗi job.

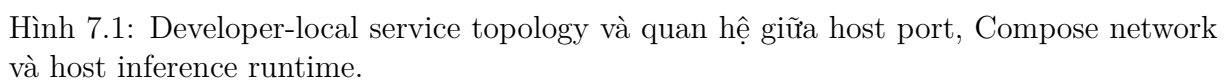
7.3 Metrics

Các nhóm metric quan trọng:

- HTTP request count và duration theo service/status.
- VTON job lifecycle counter và active queue depth.
- FASHN router worker request và in-flight count.
- Per-worker latency và failure count.
- LLM token throughput, latency và error rate.
- Guardrail block/redaction counters.
- Quota admission và remaining capacity.
- Hardware usage từ host exporter và GPU exporter.

7.4 Developer-Local Ports

Trong developer-local stack, nhiều port được expose để debug nhanh. Với shared datacenter, thiết kế nên thu hẹp public surface: gateway, Keycloak và Grafana dành cho operator; database, object store, metrics backend, guardrail và model endpoint nằm trong private network.



Bảng 7.2: Developer-local make app-up ports và override variables.

Service	Port	Override variable	Notes
TryOps Console + Rust gateway	18081	TRYOPS_GATEWAY_PORT	Main app URL. Nên dùng entry point này trước.
FastAPI backend direct	18080	TRYOPS_API_PORT	Direct API/docs access. Gateway thường proxy service này.
Keycloak IAM	18082	TRYOPS_KEYCLOAK_PORT	OIDC/IAM service. Container port là 8080.
Go controller	18084	TRYOPS_CONTROLLER_PORT	Webhook/control-plane service. Container port là 18082; Alertmanager gọi <code>controller:18082/alerts/webhook</code> trong Compose.
FASHN VTON router	18100	FASHN_VTON_ROUTER_PORT	Host-side VTON router được make app-up khởi động. API gọi qua <code>host.docker.internal:18100</code> .
FASHN VTON single-worker debug	18101	FASHN_VTON_PORT	Direct service dùng cho isolated debugging với make fashn-vton-service-bg . make app-up không dùng path này.
Optional local vLLM/OpenAI-compatible LLM	8000	VLLM_PORT VLLM_BASE_URL TRYOPS_LLM_BASE_URL	Host-side LLM endpoint khi self-host vLLM. Nếu TRYOPS_LLM_BASE_URL trỏ tới external provider, không cần local LLM port.
Postgres	15432	TRYOPS_POSTGRES_PORT	Database cho full Compose stack.
Valkey	16379	TRYOPS_VALKEY_PORT	Hot quota và rate-counter store.
MinIO API	19000	TRYOPS_MINIO_PORT	Object và artifact storage API.
MinIO Console	19001	TRYOPS_MINIO_CONSOLE_PORT	Browser UI cho MinIO.
MLflow	15000	TRYOPS_MLFLOW_PORT	Experiment và model tracking.
Prometheus	19090	TRYOPS_PROMETHEUS_PORT	Metrics database.
Alertmanager	19093	TRYOPS_ALERTMANAGER_PORT	Alert routing.
Grafana	13000	TRYOPS_GRAFANA_PORT	Dashboards. Local volume mới thường dùng <code>admin / admin</code> .
Loki	13100	TRYOPS_LOKI_PORT	Log store cho Grafana khi observability được bật. Container port là 3100.
Tempo	13200	TRYOPS_TEMPO_PORT	Trace store cho Grafana khi observability được bật. Container port là 3200.
Go guardrail sidecar	18093	TRYOPS_GUARDRAIL_PORT	LLM guardrail service. Thường được gọi bởi gateway/API. Container port là 18083.
OpenTelemetry gRPC	4317	TRYOPS_OTEL_GRPC_PORT	OTLP gRPC receiver.
OpenTelemetry HTTP	4318	TRYOPS_OTEL_HTTP_PORT	OTLP HTTP receiver.
OpenTelemetry metrics	8888	TRYOPS_OTEL_METRICS_PORT	Collector metrics endpoint.
OpenTelemetry health	13133	TRYOPS_OTEL_HEALTH_PORT	Collector health endpoint.
OTel bridge metrics	19122	TRYOPS_OTEL_BRIDGE_PORT	Metrics endpoint path là <code>/metrics</code> . Bridge local TryOps JSONL logs/traces vào OTLP để Grafana, Loki và Tempo đọc được.

7.5 Deployment Preconditions

Deployment hướng tới production cần có model endpoints được cấu hình, provider credentials được bảo vệ, development shortcut bị tắt và GPU readiness check rõ ràng cho VTON inference. Operational checks nên xác minh service health, authentication, quota admission, object storage, observability và model-serving readiness trước khi cho user submit inference job tốn tài nguyên.

8. Thực nghiệm và đánh giá

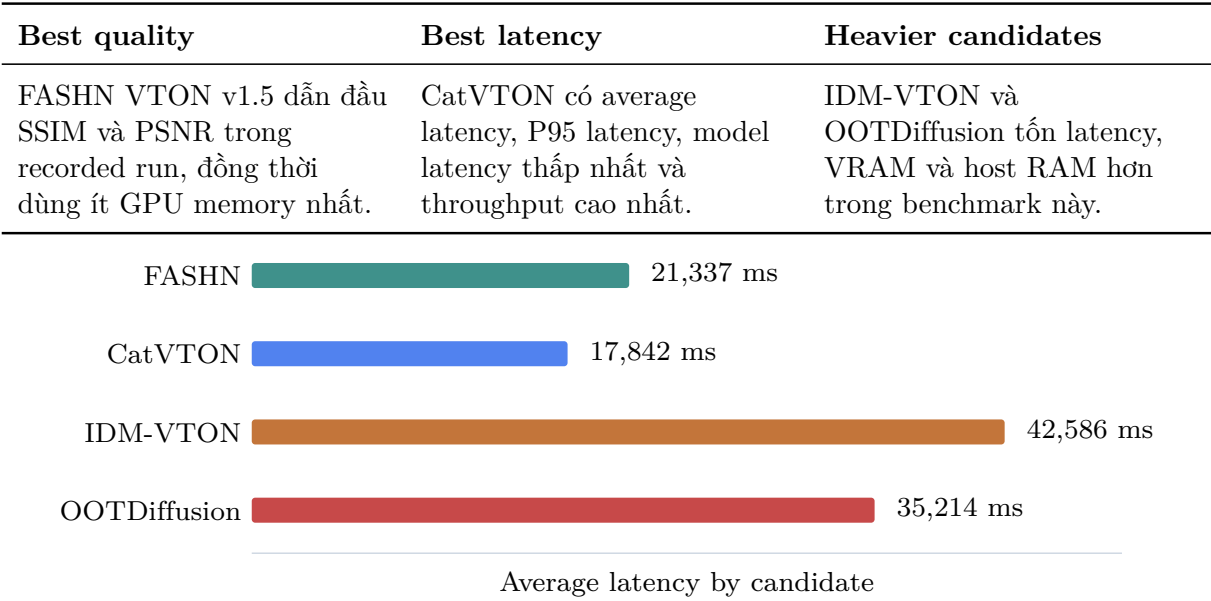
8.1 VTON Benchmark Results

Best quality score 0.888121 SSIM	Best image fidelity 18.915 PSNR	Lowest VTON memory 4.825 GB VRAM
Fastest average latency 17,842.30 ms	Highest VTON throughput 0.054261 req/s	

Bảng 8.1: Kết quả benchmark VTON dùng cho model selection.

Candidate	SSIM	PSNR	Avg ms	P95 ms	req/s	Model ms	GPU %	VRAM GB
FASHN VTON v1.5	0.888121	18.915	21,337.50	23,244.00	0.045439	19,969.60	100.00	4.825
CatVTON	0.861734	18.284	17,842.30	19,604.00	0.054261	16,512.80	96.50	6.742
IDM-VTON	0.879462	18.671	42,586.70	47,918.00	0.022751	40,318.40	99.20	13.684
OOTDiffusion	0.852918	17.936	35,214.90	39,876.00	0.027502	33,087.60	98.40	10.927

Cách đọc kết quả. FASHN VTON v1.5 là candidate mạnh nhất theo hướng ưu tiên quality vì có SSIM, PSNR và VRAM tốt nhất trong bảng. CatVTON là candidate mạnh nhất theo hướng latency/throughput, nhưng quality metrics thấp hơn.



Hình 8.1: Diễn giải benchmark VTON và so sánh average latency.

8.2 LLM Optimization Results

LLM optimization summary ghi lại Pareto frontier giữa memory và decode speed:

Bảng 8.2: Các đo đạc LLM optimization đáng chú ý.

Mode	VRAM	Decode speed	Nhận xét
fp16	1.010 GB	27.4 tokens/s	Decode speed cao nhất trong recorded run.
8-bit	0.647 GB	5.9 tokens/s	Memory thấp hơn fp16, nhưng chậm hơn trong lần đo này.
4-bit	0.478 GB	13.9 tokens/s	Low-memory option tốt hơn 8-bit trong bảng so sánh này.

Bài học vận hành là lower precision không tự động đồng nghĩa với tổng chi phí thấp hơn. Nếu quantized model decode chậm hơn nhiều trong khi board power tương tự, joules per token và rental cost per million tokens có thể tăng.

8.3 Native Boundary Evaluation

Go load driver ghi nhận Rust /health đạt 24,840.8 requests/s, trong khi FastAPI đạt 1,539.4 requests/s trên cùng máy, chênh lệch khoảng 16.1x. Điều này không có nghĩa là loại bỏ Python. Python vẫn phù hợp cho ML workflow, còn Rust/Go/C++ hỗ trợ edge, control plane và native tooling.

8.4 Balancer Testing

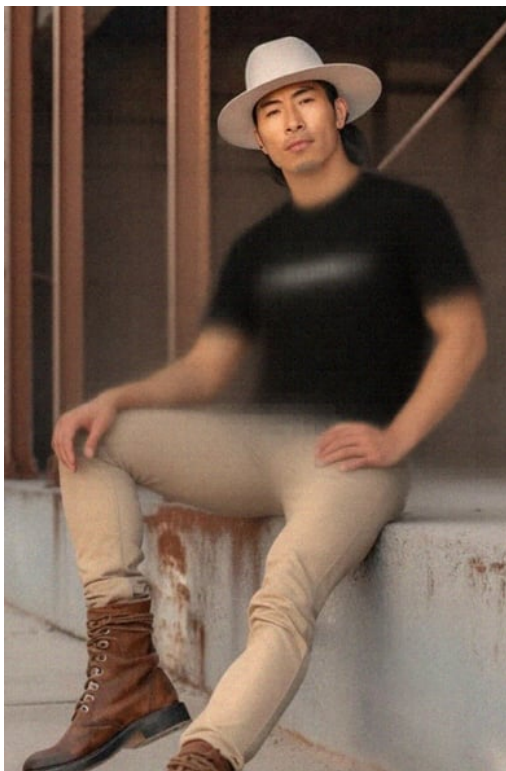
Kiểm thử router multi-GPU nên gửi nhiều request đồng thời vào router và xác minh completed-request counters tăng trên nhiều worker/GPU label. Nếu chỉ một GPU nhận việc, các điểm cần kiểm tra trước gồm workspace job concurrency, API job worker count, router worker readiness, GPU assignment và số lượng request đồng thời có đủ để vượt một worker in-flight hay không.

9. Giới hạn và hướng phát triển

9.1 Giới hạn hiện tại

- Chưa thể hot-reload để tự động deploy model mới dựa trên MLFlow.
- Live vLLM benchmark phụ thuộc host environment và serving runtime đang được cài đặt.
- KServe/Kubeflow cluster deployment là target profile, chưa phải local stack đang đóng gói.
- Tối ưu quá mạnh VTON model có thể tạo output bị mờ, mất texture và không đạt yêu cầu thị giác; VTON deployment cần image-quality regression tests, không chỉ kiểm tra API success.
- Một số operator workflow trong UI cần thêm click-through detail và export feature.
- Shared-datacenter port reduction cần hoàn thiện bằng cách đưa internal services vào ingress/private network.
- GPU utilization dashboard cần host GPU exporter và per-worker router metrics được scrape ổn định.

Một giới hạn cụ thể quan sát được khi tối ưu VTON là runtime setting quá mạnh có thể giữ được API success nhưng làm giảm chất lượng ảnh. Output có thể mờ ở vùng cơ thể và trang phục, texture không rõ, đường biên trang phục yếu. Vì vậy VTON promotion không thể chỉ dựa vào HTTP success, latency và GPU memory; cần thêm quality gate cho ảnh sinh ra.



Hình 9.1: Giới hạn khi tối ưu VTON: output bị mờ ở vùng trang phục và cơ thể, cho thấy optimized variant cần image-quality gate trước khi promotion.

9.2 Hướng phát triển

1. Có thể hot-reload để tự động deploy model mới dựa trên MLFlow gateway, và có thuật toán promotion gate reload để hạn chế downtime.
2. Harden shared-host deployment với public port surface tối thiểu.
3. Tích hợp host GPU exporter và Grafana GPU worker dashboard.
4. Tích hợp production secret manager.
5. Mở rộng VTON evaluation bằng human preference labels và fairness slices.
6. Bổ sung compatibility tests cho nhiều LLM provider/model.
7. Đóng gói KServe/Kubeflow profile cho cluster-grade deployment.
8. Cải thiện incident response runbook và dashboard drilldown.

10. Kết luận

TryOps trình bày một nền tảng MLOps xem model như production asset có quản trị, thay vì chỉ là output của notebook. Đề án tích hợp product console, model-serving boundary, account/quota control, artifact storage, model governance, observability, native control-plane components và readiness checks.

Kết quả quan trọng nhất là kiến trúc: VTON và LLM có thể dùng chung một operational spine nhưng vẫn giữ nhu cầu riêng của từng workload. VTON cần GPU-aware job control, image artifact lineage và image-quality assessment. LLM serving cần token accounting, endpoint health, guardrail và latency/cost measurement. TryOps dùng cùng gateway, API, observability, storage và governance foundation cho cả hai.

Hệ thống vẫn cần được harden thêm trước khi vận hành như một multi-tenant data-center platform. Tuy vậy, thiết kế hiện tại đã thể hiện các phần cốt lõi của MLOps: model change phải đi qua promotion gate, operator có đủ logs/metrics/traces để debug, artifact và lineage được lưu lại, còn các utility dùng cho testing/evaluation được tách khỏi product path.

Tài liệu tham khảo

- [1] OpenTelemetry, “OpenTelemetry Documentation,” [Online]. Available: <https://opentelemetry.io/docs/>. Accessed: July 4, 2026.
- [2] Prometheus Authors, “Prometheus Documentation,” [Online]. Available: <https://prometheus.io/docs/>. Accessed: July 4, 2026.
- [3] Grafana Labs, “Grafana Documentation,” [Online]. Available: <https://grafana.com/docs/grafana/latest/>. Accessed: July 4, 2026.
- [4] Grafana Labs, “Grafana Loki Documentation,” [Online]. Available: <https://grafana.com/docs/loki/latest/>. Accessed: July 4, 2026.
- [5] Grafana Labs, “Grafana Tempo Documentation,” [Online]. Available: <https://grafana.com/docs/tempo/latest/>. Accessed: July 4, 2026.
- [6] FastAPI, “FastAPI Documentation,” [Online]. Available: <https://fastapi.tiangolo.com/>. Accessed: July 4, 2026.
- [7] MLflow, “MLflow Documentation,” [Online]. Available: <https://mlflow.org/docs/latest/>. Accessed: July 4, 2026.
- [8] MinIO, “MinIO Object Storage Documentation,” [Online]. Available: <https://min.io/docs/minio/linux/>. Accessed: July 4, 2026.
- [9] Keycloak, “Keycloak Documentation,” [Online]. Available: <https://www.keycloak.org/documentation>. Accessed: July 4, 2026.
- [10] FASHN, “FASHN VTON 1.5 Research,” [Online]. Available: <https://fashn.ai/research/vton-1-5>. Accessed: July 4, 2026.
- [11] vLLM Project, “vLLM Documentation,” [Online]. Available: <https://docs.vllm.ai/>. Accessed: July 4, 2026.
- [12] OpenAI, “OpenAI API Documentation,” [Online]. Available: <https://platform.openai.com/docs/>. Accessed: July 4, 2026.
- [13] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework,” [Online]. Available: <https://www.nist.gov/itl/ai-risk-management-framework>. Accessed: July 4, 2026.

-
- [14] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications,” [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>. Accessed: July 4, 2026.
- [15] OpenSSF, “Supply-chain Levels for Software Artifacts,” [Online]. Available: <https://slsa.dev/>. Accessed: July 4, 2026.